

ЗВІТ ДО ЛАБОРАТОРНОЇ РОБОТИ 2

Реалізація алгоритмів генерації ключів гібридних криптосистем

Мова програмування Python

1 Мета роботи

Дослідити криптографічну випадковість, генерацію ключів і простих чисел та життєвий цикл ключів. Порівняти PRNG і CSPRNG та час генерації RSA-2048 і RSA-4096 у Python.

2 Основи генерації ключів

1. Ентропія характеризує невизначеність джерела для нападника. Джерелами є фізичний шум і непередбачувані системні події; ОС збирає та обробляє ці дані .

2. TRNG використовує фізичний випадковий процес, PRNG – детермінований алгоритм. CSPRNG має забезпечувати непередбачуваність результату за належної ініціалізації.

3. Seed визначає початковий стан генератора. Повтор seed відтворює послідовність; передбачуваний seed дає змогу перебрати можливі стани та відновити ключі.

4. Звичайний PRNG, наприклад `random.Random`, придатний для моделювання. Можливість відновлення його стану та наступних значень робить його непридатним для ключів.

5. У Python використовують `secrets.token_bytes` або `os.urandom`. Вони звертаються до системного джерела криптографічної випадковості; приклади API ОС – `getrandom` у Linux і `BCryptGenRandom` у Windows .

6. Симетричний ключ – випадкові байти потрібної довжини. Асиметричний ключ створюють за правилами алгоритму: для RSA генерують прості числа й пов'язані відкриті та закриті параметри.

7. Довжина AES-256 не гарантує 256 біт ентропії: якщо ключ походить із малого набору можливих seed, нападник перебирає цей набір. Гешування не додає відсутньої ентропії.

8. Після генерації потрібні контроль доступу, захищене зберігання, правильне використання, облік строку дії, ротація, відкликання та знищення. Витік ключа нівелює якість генератора.

3 Прості числа для RSA

RSA використовує великі прості p і q для модуля $n = p \times q$ та побудови ключової пари. Стійкість пов'язана зі складністю факторизації модуля.

Імовірнісні тести швидко відсіюють складені кандидати. Ферма перевіряє, чи a^u у степені $n - 1$ дає остачу 1 за модулем n . Числа Кармайкла проходять тест для всіх взаємно простих із ними основ; приклад – 561. Сильніші тести перевіряють додаткові співвідношення.

Міллер–Рабін розкладає $n - 1$ на добуток степеня двійки й непарного числа, виконує модульне піднесення до степеня та послідовні квадрати. Невідповідність умовам доводить складеність. Повтори з незалежними випадковими основами зменшують ризик помилки.

Таблиця 1 – Порівняння тестів простоти

Тест	Надійність для складеного числа	Застосування
Ферма	Можливі псевдопрості; проблема чисел Кармайкла	Лише попередня перевірка
Соловея–Штрассена	Імовірність проходження одного раунду $\leq 1/2$; критерій Ейлера та символ Якобі	Переважно історичне значення
Міллера–Рабіна	Імовірність проходження одного раунду $\leq 1/4$; сильніший критерій	Практична генерація простих чисел

Для фіксованого непарного складеного числа та незалежних рівномірних основ 64 раунди Міллера–Рабіна дають межу 2^{-128} . Це ймовірність пройти тест за умови складеності, а не ймовірність складеності прийнятого кандидата.

Мізерний ризик прийнятний за обґрунтованого вибору параметрів. Крім довжини, вимагають різних, непередбачуваних і не надто близьких p та q без повторів між ключами. Числа $p - 1$ і $q - 1$ мають бути взаємно простими з показником e .

4 Дослідження PRNG і CSPRNG

Середовище: Python 3.12.14, cryptography 50.0.1, macOS ARM64.

Програма двічі запускає дослід в окремих процесах. У кожному процесі PRNG повторно ініціалізується seed 2026, а CSPRNG формує три ключі по 32 байти. Основний фрагмент:

```
g = random.Random(2026)
a = [g.randrange(1000) for _ in range(8)]
g.seed(2026)
b = [g.randrange(1000) for _ in range(8)]
keys = [secrets.token_bytes(32) for _ in range(3)]
```

```
(venv) ~/PycharmProjects/proj_1/variant25_flask_mongo 2/lab2_project 2 % python3 lab2.py all
ЗАПУСК 1: python lab2.py rng --output results/run_1.json
PID=70649 | seed=2026 | Python 3.12.0
PRNG 1: [121, 327, 514, 974, 524, 662, 880, 975]
PRNG 2: [121, 327, 514, 974, 524, 662, 880, 975]
PRNG після повторного seed: збіг
CSPRNG: 3 навчальні ключі по 32 байти; не використовувати далі
K1: 29b11b18baf03e0523d196e73f64d021e7792df9de964e25aa1749add616342d
K2: f62600351f6cfa14217a786c6ab0d5566561aeb5d0ea968d5477cdc9df592db9
K3: a869e022f771fec4db9331f1633c128409385fc1ec26e0f7af3a68a089cf5f17

ЗАПУСК 2: python lab2.py rng --output results/run_2.json
PID=70650 | seed=2026 | Python 3.12.0
PRNG 1: [121, 327, 514, 974, 524, 662, 880, 975]
PRNG 2: [121, 327, 514, 974, 524, 662, 880, 975]
PRNG після повторного seed: збіг
CSPRNG: 3 навчальні ключі по 32 байти; не використовувати далі
K1: 1852d601e8ea16d953fa960bba51f98de86372c4b030d82aa1b20728e66e7851
K2: 730bc175c631e4ef0ef947d242669cfcc7ba2642685cae9f2b2a1b711654cecf
K3: 555f4145e491d3e1e46723549af22695808ecc50a037fe6aa29fff083be9331e

PRNG між запусками: однаковий
CSPRNG: спільних ключів між запусками 0; різних загалом 6/6
Різні значення самі по собі не доводять криптостійкість.
```

Рисунок 1 – Запис консольного виводу двох запусків

PRNG відтворив однакові 8 чисел усередині процесів і між ними. У CSPRNG отримано 6 різних ключів; збігів між запусками немає. secrets використовує джерело ОС, без заданого програмою seed. Відсутність збігів у цій малій вибірці не доводить криптостійкість. Опубліковані ключі є лише навчальними зразками.

5 Генерація RSA та вимірювання часу

Виконано по 5 генерацій для 2048 і 4096 біт, $e = 65537$. Час виміряно `perf_counter` лише навколо генерації. Поза виміром кожену пару перевірено підписом RSA-PSS і його верифікацією; усі 10 перевірок успішні. Закриті ключі не збережено. Фрагмент усередині циклу :

```
start = perf_counter()
key = rsa.generate_private_key(
    public_exponent=65537, key_size=bits)
elapsed_ms = (perf_counter() - start) * 1000
```

```
ЗАПУСК: python lab2.py rsa --trials 5
Python 3.12.0 | cryptography 50.0.1
Darwin arm64 | e=65537 | 5 повторів
Час лише generate_private_key; перевірка підпису поза виміром.

RSA-2048
Спроба 1: 51.502 мс | перевірка пари ОК
Спроба 2: 42.376 мс | перевірка пари ОК
Спроба 3: 70.587 мс | перевірка пари ОК
Спроба 4: 27.132 мс | перевірка пари ОК
Спроба 5: 39.996 мс | перевірка пари ОК
Середнє: 46.319 мс; медіана: 42.376 мс

RSA-4096
Спроба 1: 206.165 мс | перевірка пари ОК
Спроба 2: 689.498 мс | перевірка пари ОК
Спроба 3: 583.525 мс | перевірка пари ОК
Спроба 4: 1473.717 мс | перевірка пари ОК
Спроба 5: 491.256 мс | перевірка пари ОК
Середнє: 688.832 мс; медіана: 583.525 мс

Відношення середніх RSA-4096/RSA-2048: 14.87
Перевірено 10/10 пар; приватні ключі не збережено.
```

Рисунок 2 – Запис консольного виводу генерації RSA

Таблиця 2 – Підсумки п'яти вимірювань у мілісекундах

RSA, біт	Мінімум–максимум	Середнє	Медіана
2048	27.132–70.587	46.319	42.376
4096	206.165–1473.717	688.832	583.525

6 Аналіз результатів і ситуацій безпеки

Середня генерація RSA-4096 була у 12,27 раза довшою. Час коливається через різну кількість кандидатів до знаходження простих чисел і навантаження ОС. П'ять повторів дають лише орієнтовну оцінку для цього середовища.

Більший модуль збільшує вартість багаторозрядної арифметики під час генерації, підпису й розшифрування. Час підпису й розшифрування тут не вимірювався. CSPRNG забезпечує непередбачуваність кандидатів, тести простоти – їх відбір. Бібліотека зменшує ризики помилок параметрів і побічних каналів.

1. SHA-256 від часу дає передбачуваний ключ: перебір часових значень відновлює його. Слід генерувати 32 байти CSPRNG, оскільки гешування не додає ентропії.

2. Однаковий seed на IoT-пристроях повторює стан генератора й ключі. Потрібні незалежна системна ентропія на кожному пристрої, перевірка готовності джерела та заміна раніше створених слабких ключів.

3. Правильний ключ у журналі стає доступним читачам журналу: якість генерації вже не допомагає. Слід прибрати секрети з логування, обмежити доступ до копій журналу, відкликати й замінити ключ та перевірити наслідки витоку.

4. Недостатня ентропія RSA спричиняє повторні або пов'язані прості числа. Спільний множник різних модулів знаходиться через НСД і розкриває закриті ключі. Слід виправити джерело ентропії, перевірити ключі на спільні множники та перевипустити уражені пари.

5. Один довготривалий master key для всіх робить один витік небезпечним для всіх користувачів. Потрібні окремі ключі даних, розділення доступу, версії ключів і планова ротація. Ротація обмежує подальші ризики, але не скасовує вже здійсненого розкриття.

7 Життєвий цикл ключового матеріалу

Таблиця 3 – Загрози та вимоги за етапами

Етап	Основна загроза	Вимога
Генерація	Слабкий seed, повтори ключів	CSPRNG ОС, перевірені параметри та бібліотека
Зберігання	Копіювання секрету, втрата доступу	Сховище секретів, мінімальні права, захищені резервні копії за потреби
Використання	Логи, помилки API, надмірні права	Обмежити призначення ключа; не журналювати секрет; контролювати доступ
Ротація або заміна	Надто довгий строк дії, втрата старих даних	Визначити криптоперіод, версіонувати ключі та керувати переходом
Відкликання або знищення	Подальше прийняття скомпрометованого ключа, залишкові копії	Заборонити нове використання; за потреби повідомити сторони; знищити копії після завершення потреби

Відкликання припиняє довіру до ключа, а знищення усуває його матеріал. Це різні дії: для розшифрування архіву старий ключ іноді зберігають із вузькими правами; скомпрометованим ключем нові дані не захищають.

Висновки

Підтверджено відтворюваність PRNG за однакового seed і отримано шість різних 256-бітових зразків через CSPRNG ОС. Згенеровано й перевірено 10 RSA-пар; у цьому досліді RSA-4096 потребував суттєво більше часу. Безпека визначається не лише довжиною ключа: потрібні достатня ентропія, надійна бібліотека та захист ключа на всіх етапах його життєвого циклу.